

*A*  
***Project Report***  
*on*  
**CIFAKE Image Classification and Explainable Identification of  
AI-Generated Synthetic Images**

*Submitted to*  
**XtraGrad Pvt. Ltd**

**AICTE INTERNSHIP**

**By**

**“Niraj Khankari  
AICTE STU ID: STU6867ed37a45b01751641399 ”**

Under the guidance of  
**Abhienaya Sri Soundarajan**

**Project Github Link :**  
**[https://github.com/NirajKhankari30/CIFAKE-Image-Classification-and-Explainable-Id  
entification-of-AI-Generated-Synthetic-Images](https://github.com/NirajKhankari30/CIFAKE-Image-Classification-and-Explainable-Identification-of-AI-Generated-Synthetic-Images)**

# INDEX

## CHAPTER 1: INTRODUCTION

- Project Background
- Problem Statement
- Objectives
- Scope of the Project
- Expected Outcomes

## CHAPTER 2: SYSTEM OVERVIEW

- Existing System
- Proposed System
- System Modules
- Overall Workflow

## CHAPTER 3: LITERATURE SURVEY

- Research Papers Reviewed
- Existing Solutions
- Comparative Analysis
- Research Gap

## CHAPTER 4: TECHNOLOGIES USED

- Programming Languages
- Frameworks and Libraries
- Development Tools

## CHAPTER 5: ARCHITECTURE AND DESIGN

- System Architecture Diagram
- Workflow Explanation

## CHAPTER 6: IMPLEMENTATION

- Key Features Implemented

## CHAPTER 7: RESULTS AND DISCUSSION

- Screenshots
- Output Analysis
- Performance Evaluation
- Testing Results

## CHAPTER 8: CHALLENGES AND LEARNING OUTCOMES

- Challenges Faced
- Solutions Implemented
- Technical Skills Learned

## CHAPTER 9: FUTURE ENHANCEMENTS

- Proposed Improvements
- Additional Features
- Scalability Opportunities

## CHAPTER 10: CONCLUSION

- Summary of Work Completed
- Achievements
- Project Impact
- Final Remark

# CHAPTER 1: INTRODUCTION

## 1.1 Project Background

- Growth of AI-generated images
- Increasing realism of synthetic images
- Difficulty distinguishing AI-generated and real images manually
- Importance of automated image classification
- Introduction to the CIFAKE dataset
- Use of CNN and Explainable AI

## 1.2 Problem Statement

The increasing availability of generative artificial intelligence has made it possible to create highly realistic synthetic images that can be difficult to distinguish from real images. This creates a need for an automated system that can classify images as real or AI-generated and provide an understandable explanation for its prediction.

The project addresses this problem using a CNN-based binary image classifier with Grad-CAM-based visual explanation.

## 1.3 Objectives

The objectives of the project are:

- To study and prepare the CIFAKE image dataset.
- To validate the dataset and identify invalid or corrupted images.
- To preprocess images for CNN-based classification.
- To apply image augmentation to improve model generalization.
- To develop a CNN model for FAKE/REAL image classification.
- To evaluate the model using accuracy, precision, recall and F1-score.
- To analyze incorrectly classified and low-confidence images.
- To implement Grad-CAM for visual interpretation of CNN predictions.
- To save and document the trained model and evaluation results.
- To develop a foundation for future AI-generated image detection systems.

## 1.4 Scope of the Project

The project focuses on:

- Binary classification of images into FAKE and REAL.
- CNN-based image classification.
- Dataset preprocessing and validation.
- Model performance evaluation.
- Misclassification analysis.
- Confidence analysis.
- Explainable AI using Grad-CAM.

## **1.5 Expected Outcomes**

- A validated image dataset.
- A trained CNN classification model.
- Automated FAKE/REAL predictions.
- Quantitative model evaluation.
- Confusion matrix and classification report.
- Misclassified-image analysis.
- Confidence/uncertainty analysis.
- Grad-CAM visual explanations.
- A saved trained model that can be reused for prediction.

# CHAPTER 2: SYSTEM OVERVIEW

## 2.1 Existing System

Traditional identification of AI-generated images often depends on:

- Human visual inspection
- Manual analysis of image artifacts
- Metadata examination
- Hand-crafted image features
- Conventional machine-learning techniques

Human inspection can become difficult when generated images are highly realistic. Traditional approaches may also require manually designed features and may not generalize well to different types of images.

Deep learning provides an alternative by learning visual patterns directly from training images.

## 2.2 Proposed System

The proposed system uses a Convolutional Neural Network (CNN) to classify images into two categories:

- FAKE — AI-generated/synthetic image
- REAL — real image

The system additionally uses Grad-CAM to visualize the regions of an image that contributed to the model's prediction.

Proposed system includes:

1. Dataset validation
2. Image preprocessing
3. Data augmentation
4. Dataset splitting
5. CNN model training
6. Model evaluation
7. Prediction analysis
8. Grad-CAM explanation
9. Result and model saving

## 2.3 System Modules

The system consists of the following modules:

### Module 1 — Dataset Management

Handles:

- FAKE images
- REAL images
- File validation
- Dataset statistics

## **Module 2 — Data Preprocessing**

Handles:

- Image loading
- RGB conversion
- Resizing
- Pixel normalization

## **Module 3 — Data Augmentation**

Applies:

- Horizontal flipping
- Random rotation
- Random zoom

## **Module 4 — Dataset Splitting**

The dataset is divided into:

- 70% Training
- 15% Validation
- 15% Testing

## **Module 5 — CNN Classification**

The CNN learns visual features and performs binary classification.

## **Module 6 — Model Evaluation**

Evaluates:

- Accuracy
- Precision
- Recall
- F1-score
- Error rate
- Confusion matrix

## **Module 7 — Prediction Analysis**

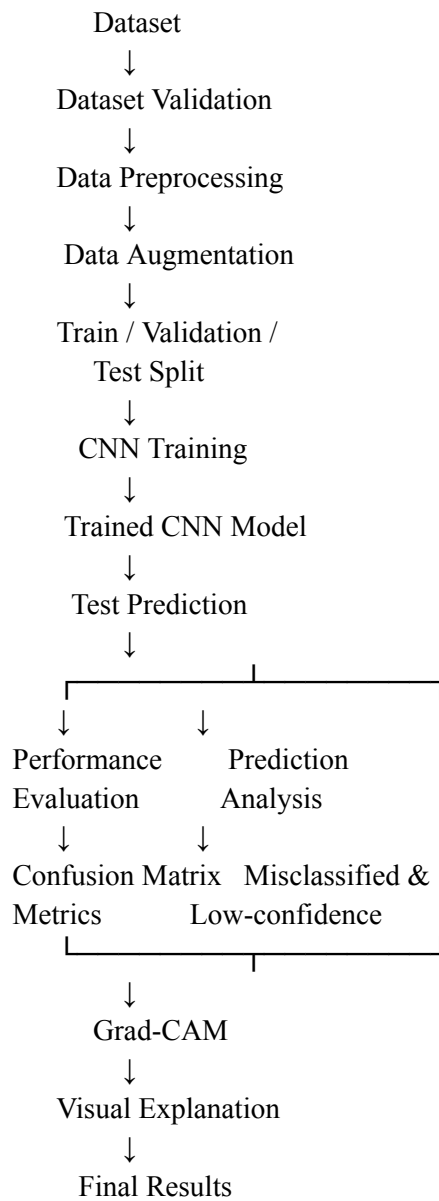
Analyzes:

- Correct predictions
- Incorrect predictions
- Prediction probabilities
- Low-confidence predictions

## Module 8 — Explainable AI

Grad-CAM is used to visualize important image regions influencing the prediction.

### 2.4 Overall Workflow



# CHAPTER 3: LITERATURE SURVEY

## 3.1 Research Papers Reviewed

- AI-generated image detection
- CNN-based image classification
- Synthetic image detection
- Deep learning for image forensics
- Explainable AI

## 3.2 Existing Solutions

- Traditional image-processing techniques
- Hand-crafted feature extraction
- Machine-learning classifiers
- CNN-based classifiers
- Transfer-learning models
- Frequency-domain analysis
- GAN-specific detection techniques
- Explainable AI-based detection systems

## 3.3 Comparative Analysis

| Approach          | Feature Extraction    | Automation | Interpretability | Limitation                                                      |
|-------------------|-----------------------|------------|------------------|-----------------------------------------------------------------|
| Manual Inspection | Human observation     | Low        | High             | Time-consuming                                                  |
| Traditional ML    | Hand-crafted features | Medium     | Medium           | Feature engineering required                                    |
| CNN               | Automatic             | High       | Low without XAI  | Can behave like a black box                                     |
| CNN + Grad-CAM    | Automatic             | High       | Higher           | Explanation is visual rather than a complete causal explanation |

|                          |           |      |                 |                             |
|--------------------------|-----------|------|-----------------|-----------------------------|
| <b>Transfer Learning</b> | Automatic | High | Low without XAI | Depends on pretrained model |
|--------------------------|-----------|------|-----------------|-----------------------------|

## 3.4 Research Gap

The project identifies the following practical gap:

Although deep-learning approaches can achieve effective image classification, a prediction alone does not explain why a particular image was classified as real or fake. Therefore, combining image classification with an explainability technique such as Grad-CAM can provide additional visual information about the regions influencing the prediction.

**CNN Classification + Performance Evaluation + Misclassification Analysis + Grad-CAM**

# CHAPTER 4: TECHNOLOGIES USED

## 4.1 Programming Languages

### Python

Python was used as the primary programming language because it provides extensive support for:

- Machine learning
- Deep learning
- Image processing
- Data analysis
- Visualization

## 4.2 Frameworks and Libraries

### TensorFlow

Used for building and training the CNN.

### Keras

Used to define:

- CNN layers
- Model architecture
- Optimizer
- Loss function
- Callbacks

### NumPy

Used for numerical operations and arrays.

### Pandas

Used for:

- Dataset organization
- Prediction results
- CSV files
- Data analysis

### Matplotlib

Used for:

- Training graphs
- Image visualization
- Grad-CAM visualization

## **Seaborn**

Used for:

- Confusion matrix visualization
- Statistical plots

## **Scikit-learn**

Used for:

- Train/test splitting
- Confusion matrix
- Classification report
- Evaluation metrics

## **PIL**

Used for:

- Image loading
- Image validation
- Image information analysis

## **4.3 Development Tools**

### **Jupyter Notebook**

Used as the primary development environment for:

- Data analysis
- Model development
- Training
- Evaluation
- Visualization
- Grad-CAM experimentation

## **Anaconda**

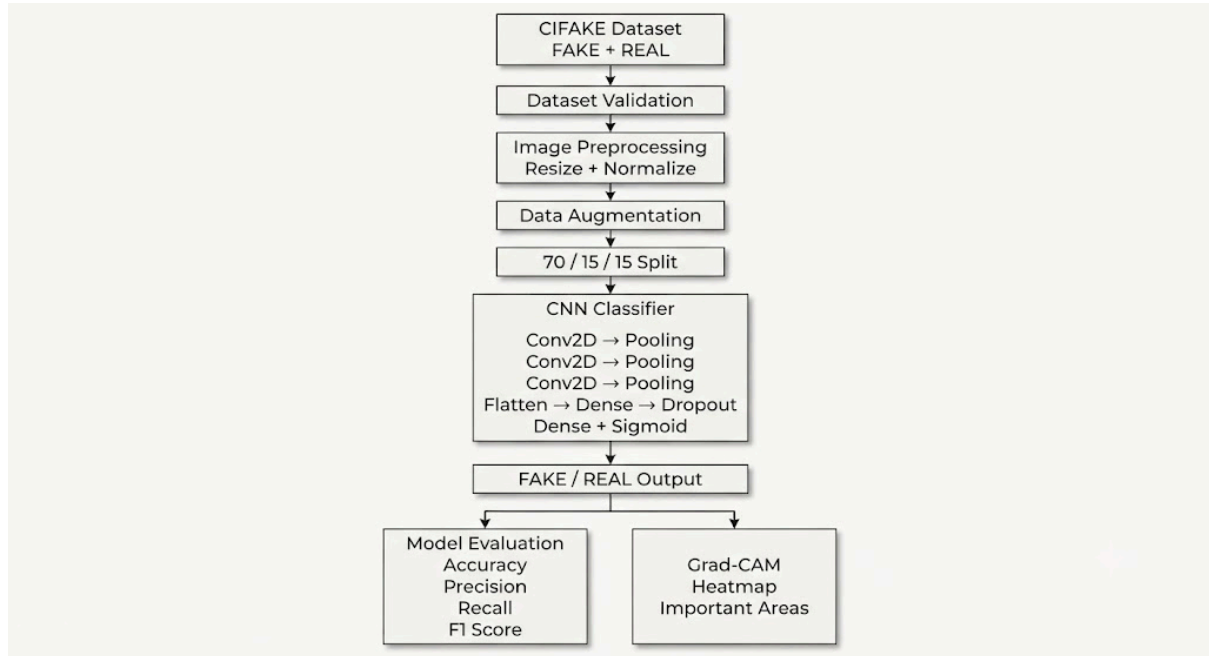
Used for Python environment and package management.

## **Windows File System**

Used for local dataset and project-result management.

# CHAPTER 5: ARCHITECTURE AND DESIGN

## 5.1 System Architecture Diagram



## 5.2 Workflow Explanation

The image first enters the dataset-validation stage. Valid images are then resized to  $32 \times 32$  pixels and converted to normalized RGB format.

Training images undergo augmentation to introduce small variations.

The CNN extracts increasingly complex visual features through convolution and pooling layers. The final sigmoid layer produces a probability associated with the REAL class.

A threshold of 0.5 is used:

Probability  $\geq 0.5 \rightarrow$  REAL

Probability  $< 0.5 \rightarrow$  FAKE

After classification, Grad-CAM generates a heatmap showing image regions associated with the model's prediction.

# CHAPTER 6: IMPLEMENTATION

## 6.1 Key Features Implemented

### 6.1.1 Dataset Validation

- Available image files
- Class distribution
- Image dimensions
- Color modes
- Corrupted images
- Valid file extensions

### 6.1.2 Dataset Preparation

Images were organized into:

- FAKE
- REAL

Labels were assigned:

- 0 → FAKE
- 1 → REAL

### 6.1.3 Image Preprocessing

- Loaded from disk
- Decoded
- Converted to RGB
- Resized to  $32 \times 32$
- Converted to floating-point values
- Normalized

### 6.1.4 Data Augmentation

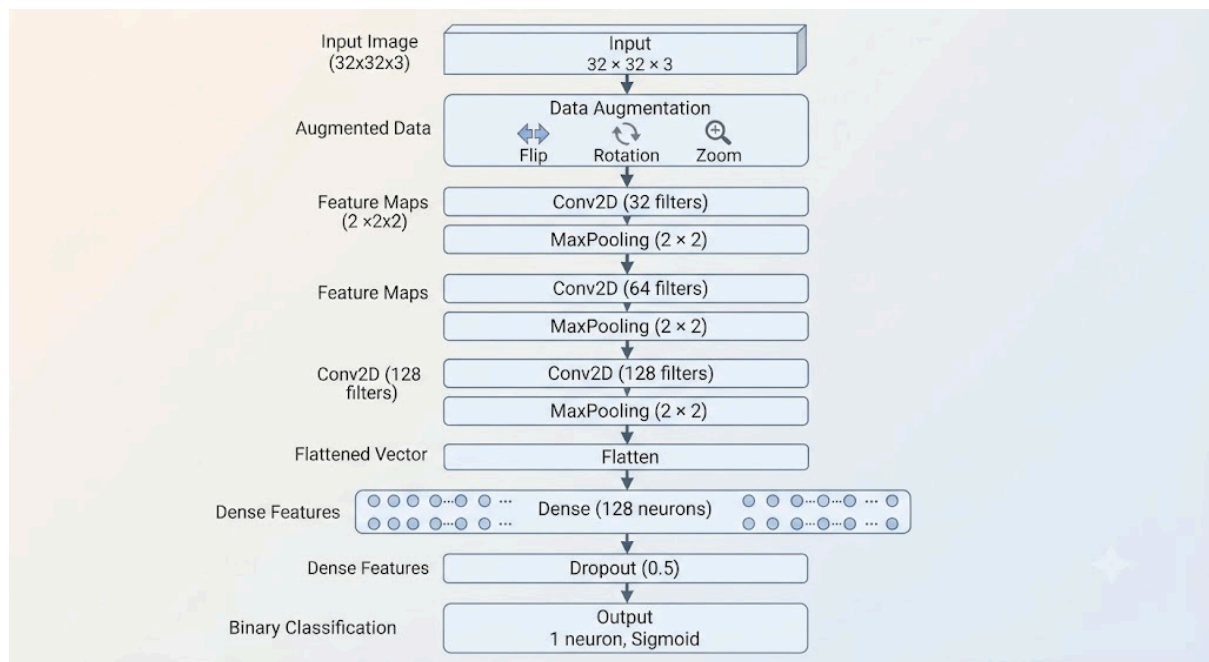
- Random Horizontal Flip
- Random Rotation
- Random Zoom

### 6.1.5 Dataset Split

- Training → 70%
- Validation → 15%
- Testing → 15%

### 6.1.6 CNN Model

The implemented CNN consists of:



### 6.1.7 Model Training

- Optimizer: Adam
- Learning rate: 0.001
- Loss: Binary Crossentropy
- Metric: Accuracy
- Maximum epochs: 20
- Early stopping
- Model checkpointing

The best model was selected using validation performance.

### 6.1.8 Prediction

The trained model generates a probability between 0 and 1.

The output is interpreted as the probability of the REAL class.

### 6.1.9 Performance Evaluation

- Accuracy
- Precision
- Recall
- F1-score
- Error rate
- Confusion matrix

### 6.1.10 Misclassification Analysis

Incorrectly classified images were extracted and visually analyzed.

Low-confidence predictions were also identified using the distance of the prediction probability from 0.5.

### **6.1.11 Grad-CAM Implementation**

Grad-CAM was implemented using the final convolutional layer:

`conv2d_2`

The generated visualization contains:

1. Original image
2. Grad-CAM heatmap
3. Heatmap overlay

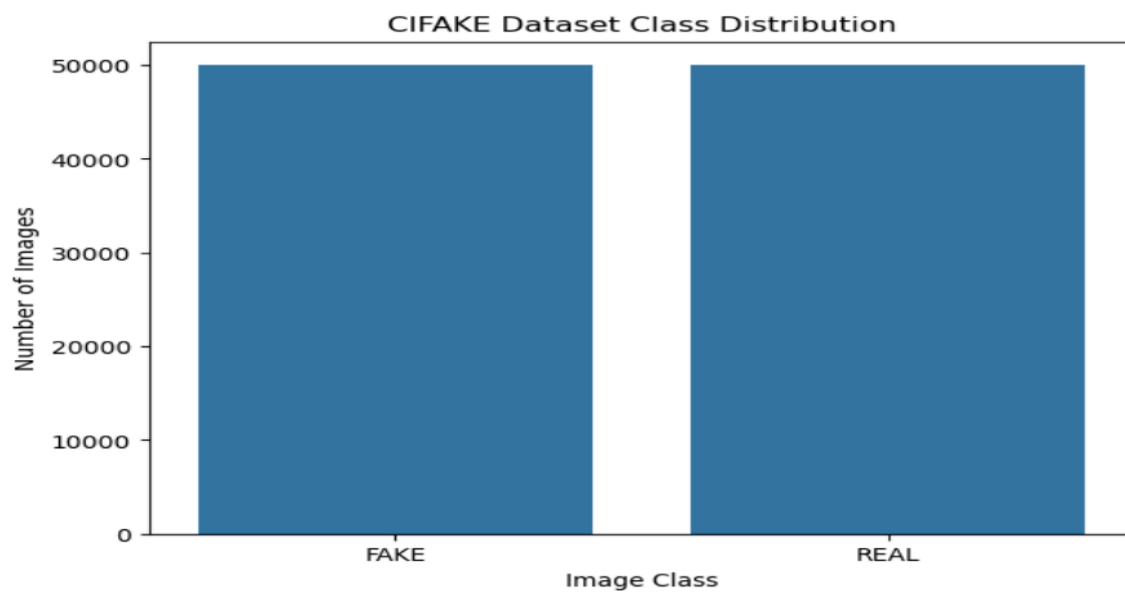
This helps visualize regions contributing to the CNN prediction.

# CHAPTER 7: RESULTS AND DISCUSSION

## 7.1 Screenshots

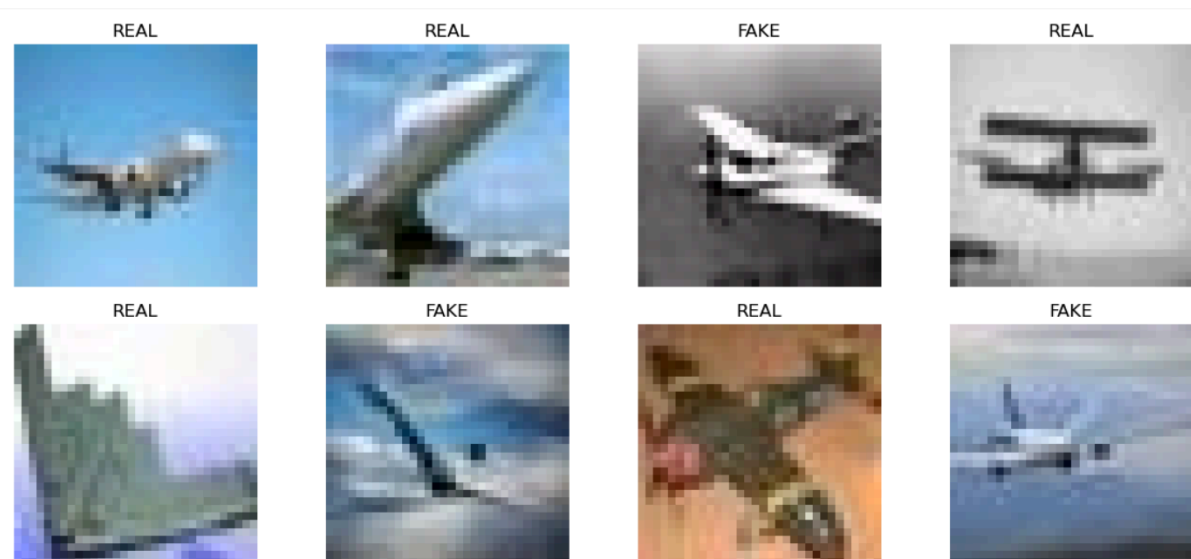
### Screenshot 1

Dataset class distribution.



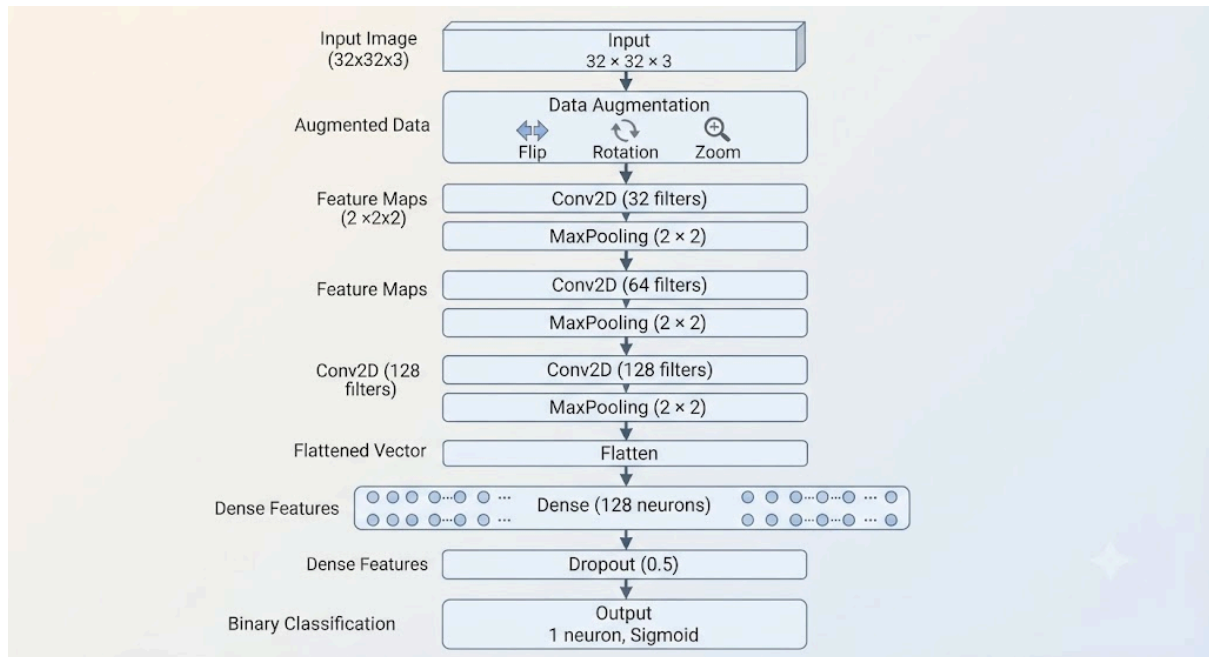
### Screenshot 2

FAKE and REAL sample images.



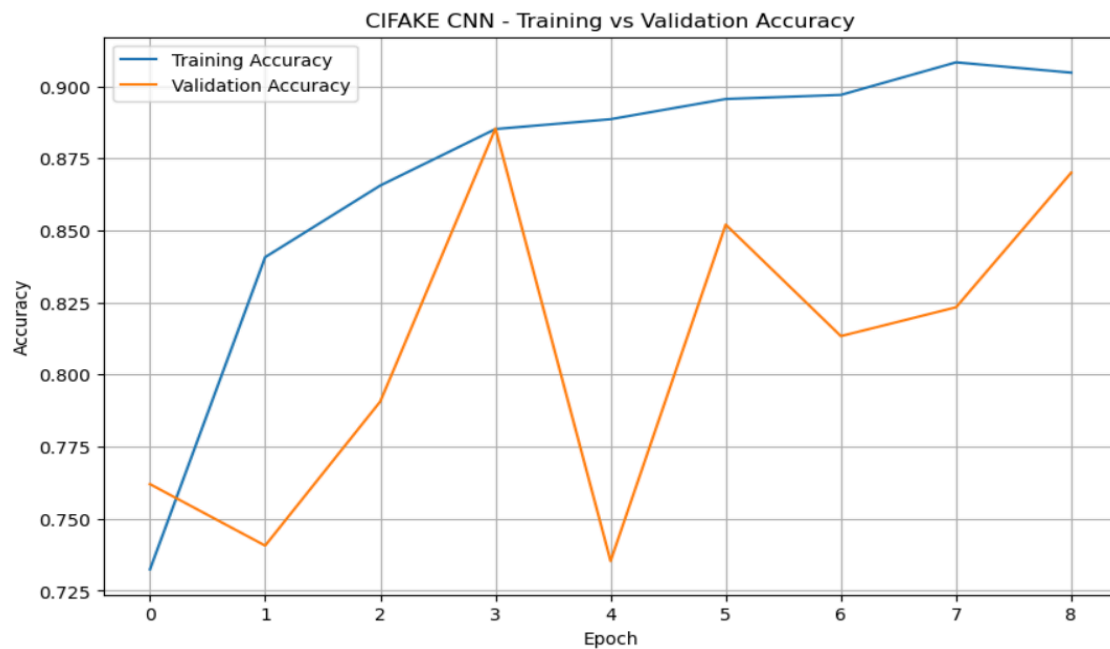
### Screenshot 3

CNN model architecture.



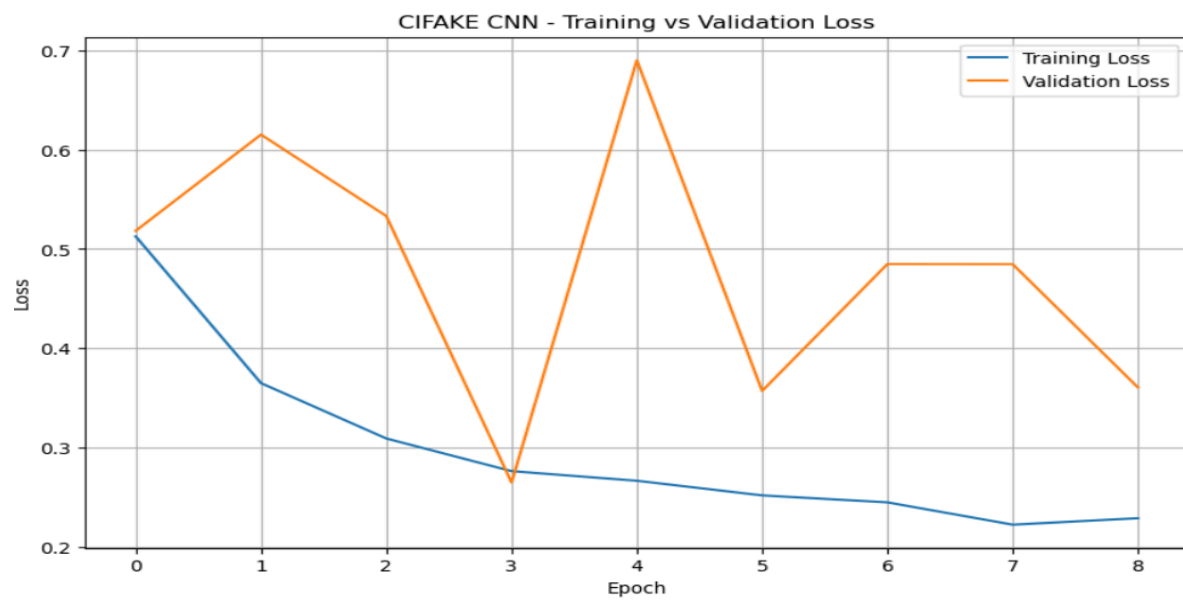
### Screenshot 4

Training accuracy and validation accuracy.



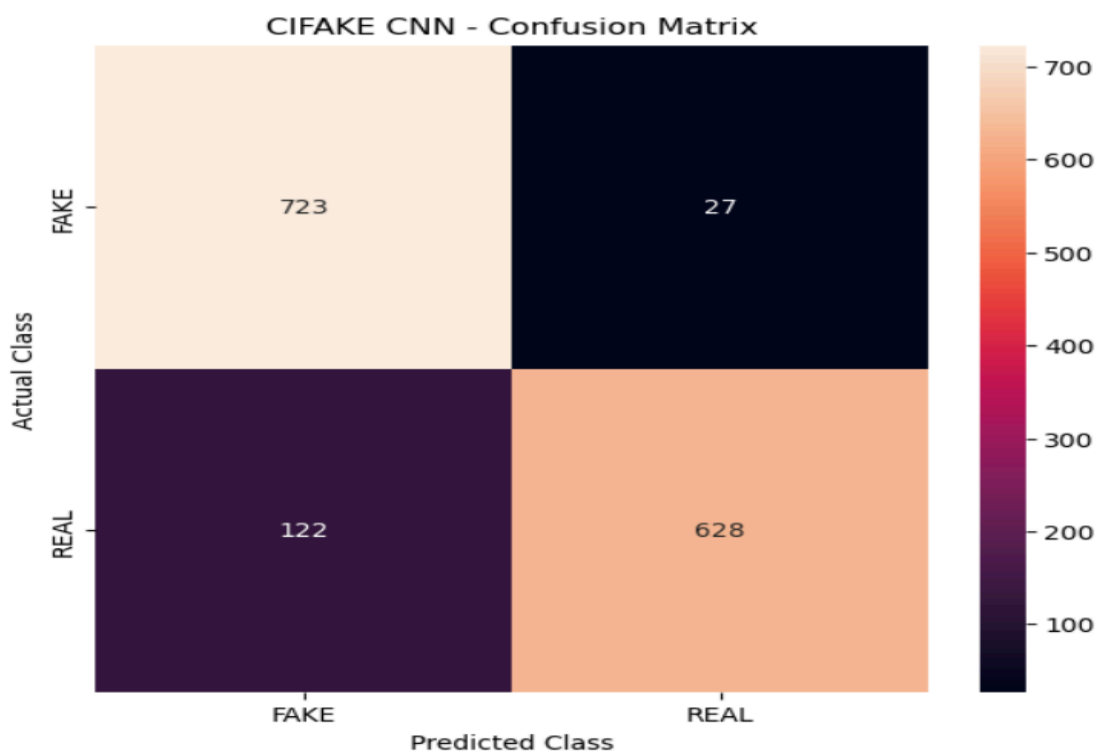
### Screenshot 5

Training loss and validation loss.



## Screenshot 6

Confusion matrix.



## Screenshot 7

Classification report.

## Classification Report

```
report = classification_report(
    y_true,
    y_pred,
    target_names=class_names,
    digits=4
)

print(report)
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| FAKE         | 0.8556    | 0.9640 | 0.9066   | 750     |
| REAL         | 0.9588    | 0.8373 | 0.8940   | 750     |
| accuracy     |           |        | 0.9007   | 1500    |
| macro avg    | 0.9072    | 0.9007 | 0.9003   | 1500    |
| weighted avg | 0.9072    | 0.9007 | 0.9003   | 1500    |

## Screenshot 8

Misclassified images.

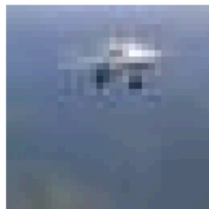
Total misclassified images: 149

|      | filepath                                          | class | label | actual_label | predicted_label | probability_real | actual_class | predicted_class |
|------|---------------------------------------------------|-------|-------|--------------|-----------------|------------------|--------------|-----------------|
| 5435 | D:\Niraj Khankari\# INTERNSHIPS\Xtragrad Inter... | REAL  | 1     | 1            | 0               | 0.002072         | REAL         | FAKE            |
| 5240 | D:\Niraj Khankari\# INTERNSHIPS\Xtragrad Inter... | FAKE  | 0     | 0            | 1               | 0.549555         | FAKE         | REAL            |
| 8239 | D:\Niraj Khankari\# INTERNSHIPS\Xtragrad Inter... | FAKE  | 0     | 0            | 1               | 0.567469         | FAKE         | REAL            |
| 4579 | D:\Niraj Khankari\# INTERNSHIPS\Xtragrad Inter... | REAL  | 1     | 1            | 0               | 0.215115         | REAL         | FAKE            |
| 9817 | D:\Niraj Khankari\# INTERNSHIPS\Xtragrad Inter... | REAL  | 1     | 1            | 0               | 0.074860         | REAL         | FAKE            |

Actual: REAL  
Predicted: FAKE  
Confidence: 99.8%



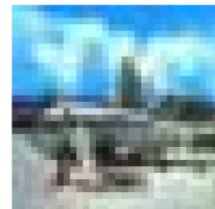
Actual: FAKE  
Predicted: REAL  
Confidence: 55.0%



Actual: FAKE  
Predicted: REAL  
Confidence: 56.7%



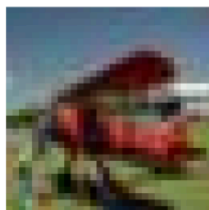
Actual: REAL  
Predicted: FAKE  
Confidence: 78.5%



Actual: REAL  
Predicted: FAKE  
Confidence: 92.5%



Actual: REAL  
Predicted: FAKE  
Confidence: 75.7%



Actual: REAL  
Predicted: FAKE  
Confidence: 54.8%



Actual: REAL  
Predicted: FAKE  
Confidence: 88.9%



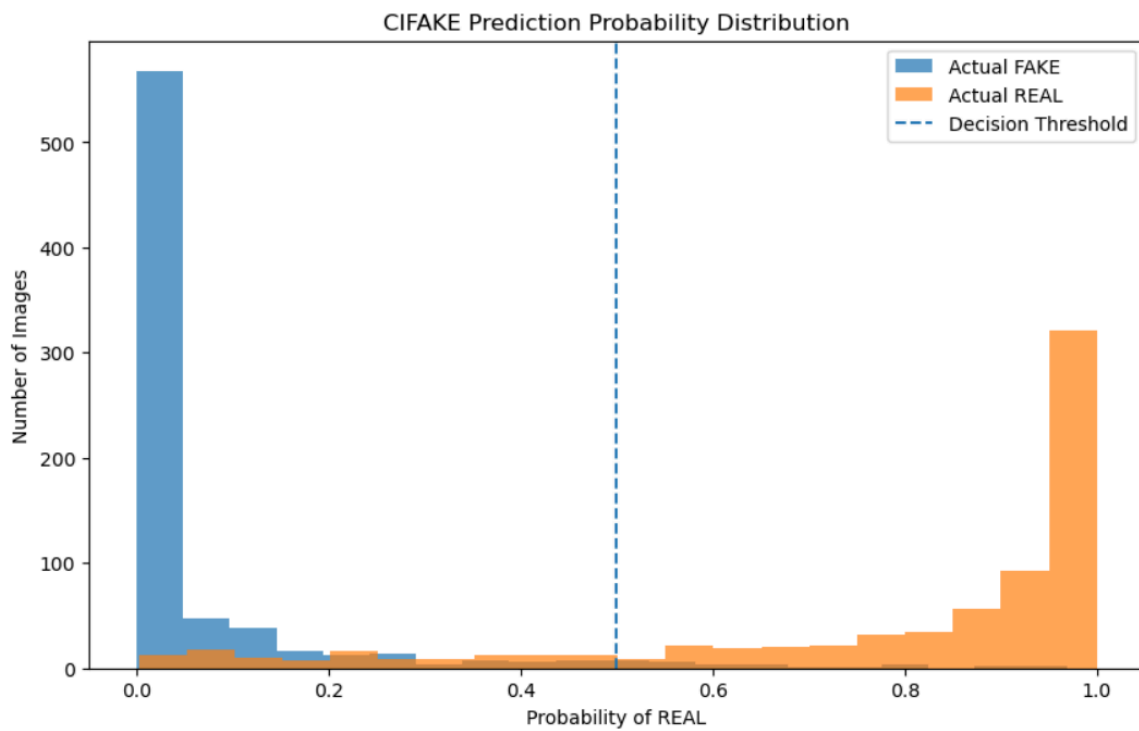
## Screenshot 9

Low-confidence predictions.



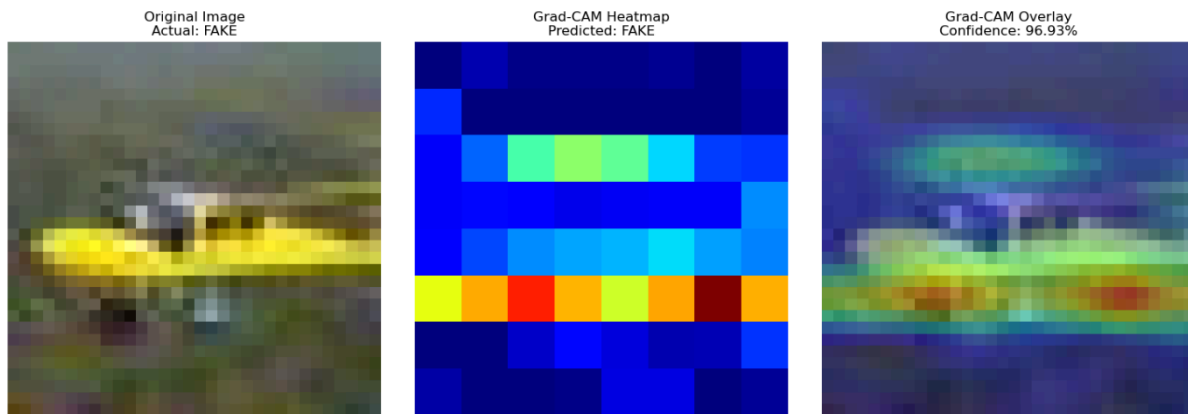
## Screenshot 10

CIFAKE Prediction Probability Distribution



## Screenshot 11

Grad-CAM visualization.



## 7.2 Output Analysis

The trained model was evaluated on 1,500 test images.

The confusion matrix obtained was:

```
-----
CONFUSION MATRIX
-----
[[723  27]
 [122 628]]

True Negatives (TN): 723
False Positives (FP): 27
False Negatives (FN): 122
True Positives (TP): 628
```

Therefore:

- True Negatives = 723
- False Positives = 27
- False Negatives = 122
- True Positives = 628

The model correctly classified:

1,351 out of 1,500 images.

## 7.3 Performance Evaluation

```
=====
FINAL CIFAKE MODEL RESULTS
=====
Total Test Images      : 1500
Correct Predictions    : 1351
Incorrect Predictions  : 149
Accuracy               : 90.07%
Precision              : 95.88%
Recall                 : 83.73%
F1-Score               : 89.40%
Error Rate             : 9.93%
Best Epoch             : 4
Best Validation Accuracy : 88.53%

✓ Training accuracy saved
✓ Training loss saved
✓ Confusion matrix image saved
✓ Confusion matrix CSV saved
✓ Classification report saved
✓ Test predictions CSV saved
✓ Final results CSV saved
✓ Grad-CAM image saved
✓ Final model copied to Results folder
```

## Interpretation

The model achieved an overall test accuracy of **90.07%**.

The precision of **95.88%** indicates that among images predicted as REAL, a high proportion were actually REAL.

The recall of **83.73%** indicates that some REAL images were classified as FAKE.

The F1-score of **89.40%** provides a combined measure of precision and recall.

The model therefore demonstrated useful performance on the evaluated CIFAKE test set, while the misclassified images indicate that distinguishing certain real and synthetic images remains challenging.

## 7.4 Testing Results

- Dataset validation testing
- Corrupted-image testing
- Model prediction testing
- Confusion-matrix evaluation
- Classification-report evaluation
- Misclassification testing
- Low-confidence prediction analysis

- Grad-CAM visualization testing

The model successfully produced binary predictions for the test dataset and generated visual explanations for selected predictions.

# CHAPTER 8: CHALLENGES AND LEARNING OUTCOMES

## 8.1 Challenges Faced

### Challenge 1 — Dataset File Handling

Some image paths required careful validation because stale filenames could cause `FileNotFoundError`.

#### Solution

The image file lists were refreshed from the actual dataset folders before processing.

### Challenge 2 — Corrupted Images

Images may be unreadable or corrupted.

#### Solution

PIL-based image verification was performed before model training.

### Challenge 3 — CNN Model Training

Selecting suitable image dimensions, layers, optimizer and training parameters required experimentation.

#### Solution

A lightweight CNN architecture was developed and combined with augmentation and early stopping.

### Challenge 4 — Grad-CAM with Keras Model Graph

The original Sequential model created graph-connectivity issues when directly attempting to construct the Grad-CAM model.

#### Solution

A Functional API graph was reconstructed using the trained layer configurations and weights. This allowed intermediate convolutional outputs to be accessed for Grad-CAM.

### Challenge 5 — Windows File Path Length

Saving some output files to the deeply nested project directory caused file-writing errors.

## **Solution**

A shorter output directory was created:

D:\CIFAKE\_Project\_Results

This simplified output management.

## **8.2 Technical Skills Learned**

- Python programming
- Dataset preprocessing
- Image processing
- CNN architecture design
- TensorFlow/Keras
- Model training
- Model evaluation
- Classification metrics
- Confusion matrix analysis
- Data visualization
- Error analysis
- Confidence analysis
- Explainable AI
- Grad-CAM
- Jupyter Notebook
- File and project management
- Machine-learning debugging

# CHAPTER 9: FUTURE ENHANCEMENTS

## 9.1 Proposed Improvements

Future versions can use larger image resolutions such as:

$64 \times 64$

$128 \times 128$

$224 \times 224$

This may allow the model to capture more detailed visual patterns.

## 9.2 Additional Features

- Transfer learning
- ResNet
- EfficientNet
- MobileNet
- Vision Transformers
- More advanced XAI techniques
- Automatic report generation
- Confidence-based warnings
- Web-based prediction interface

## 9.3 Scalability Opportunities

- Larger datasets
- Multiple image sources
- Different AI-generation methods
- Video frames
- Real-time image analysis
- Cloud-based deployment
- API-based prediction
- Web applications

Another possible extension would be multi-class classification to identify different types or sources of synthetic images rather than only FAKE/REAL classification.

# CHAPTER 10: CONCLUSION

## 10.1 Summary of Work Completed

The project developed an end-to-end deep-learning pipeline for identifying AI-generated synthetic images using the CIFAKE dataset.

The work included:

1. Dataset validation
2. Data preprocessing
3. Image augmentation
4. Stratified dataset splitting
5. CNN model development
6. Model training
7. Performance evaluation
8. Misclassification analysis
9. Confidence analysis
10. Grad-CAM-based explainability
11. Result and model saving

## 10.2 Achievements

The developed CNN achieved the following results on the test set:

- 90.07% Accuracy
- 95.88% Precision
- 83.73% Recall
- 89.40% F1-Score

The model correctly classified 1,351 out of 1,500 test images.

Grad-CAM was also successfully implemented to provide a visual interpretation of model predictions.

## 10.3 Project Impact

The project demonstrates how deep learning can be used to automate the classification of real and synthetic images while also providing visual information about the regions influencing model predictions.

This provides a practical foundation for further development of AI-generated content detection systems.

## 10.4 Final Remark

The project successfully combined Computer Vision, Deep Learning, Model Evaluation and Explainable AI into a single image-classification workflow. The implementation also provided

practical experience in dataset preparation, CNN development, debugging, performance analysis and model interpretation.